Selection Sort

Finding the Extremum: A Systematic Approach

Module 11: Ordering & Permutations

Series: CodeHive Algorithm Library

01. The Minimum-First Strategy

Selection Sort is a straightforward comparison-based sorting algorithm. The central logic is built around a "Search and Place" cycle: find the smallest (minimum) value in the unsorted portion of the array and swap it to its correct chronological position.

How it functions:

1. Scan the entire array to identify the **lowest value**.
2. Move that value to the front of the current unsorted segment.
3. Define the remaining array as the new unsorted segment and repeat until sorted.

Key Difference from Bubble Sort: While Bubble Sort continuously swaps adjacent elements, Selection Sort finds the absolute minimum first and performs exactly **one swap per pass**.

02. Manual Execution Trace

Input Array: `[7, 12, 9, 11, 3]`

The Passes:

- **Pass 1:** Search `[7, 12, 9, 11, 3]`. Minimum is 3.
  Swap 3 with 7. Result: `[3, 12, 9, 11, 7]`
- **Pass 2:** Search `[12, 9, 11, 7]`. Minimum is 7.
  Swap 7 with 12. Result: `[3, 7, 9, 11, 12]`
- **Pass 3:** Search `[9, 11, 12]`. Minimum is 9.
  No swap needed. Result: `[3, 7, 9, 11, 12]`
- **Pass 4:** Search `[11, 12]`. Minimum is 11.
  No swap needed. Sorted!

Note how each pass guarantees that the front-most part of the array reaches its final sorted state.

03. The Hidden Mutation Problem

In high-level languages like Python, it is tempting to use `pop()` and `insert()` to move the minimum value to the front. However, this hides an expensive architectural cost.

Theoretical Shifting:

When you "pop" an item from index 5 and "insert" it at index 0, the computer must shift every item between those indices to fill the gap and make space. This is a **Linear Time O(n)** operation happening inside your sort loop.

```
# Inefficient Implementation (Hidden Shifting)
for i in range(n-1):
  min_index = i
  # ... find min_index ...
  min_value = mylist.pop(min_index) # Costly shift!
  mylist.insert(i, min_value)       # Costly shift!
```

04. The In-Place Swap Solution

To avoid massive data shifts, we use an **In-Place Swap**. Instead of removing the item, we swap it with the element currently occupying its target destination.

```
mylist = [64, 34, 25, 12, 22, 11, 90, 5]
n = len(mylist)

for i in range(n):
    min_index = i
    for j in range(i+1, n):
        if mylist[j] < mylist[min_index]:
            min_index = j

    # Swap the found minimum with the first unsorted element
    mylist[i], mylist[min_index] = mylist[min_index], mylist[i]

print(mylist)
```

Memory Efficiency: This version uses $O(1)$ extra space, as it rearranges the original array without creating new lists or shifting blocks of memory.

05. Computational Analysis

Selection Sort's performance is deterministic; it always performs the same number of comparisons regardless of the initial data order.

Big O Metrics:

- **Time Complexity:** $O(n^2)$ - Because of the nested loop (searching n items, n times).
- **Space Complexity:** $O(1)$ - Constant space (No extra memory required for swaps).
- **Best/Average/Worst Case:** All are $O(n^2)$. Even if the array is already sorted, Selection Sort will still scan for the minimum every time.

> **Warning:** Because it is always $O(n^2)$, Selection Sort is generally slower than an optimized Bubble Sort on data that is mostly sorted.

06. Practical Verdict

Selection Sort is favored over more complex algorithms in specific scenarios where data movement (writing to memory) is more expensive than data comparison.

Core Takeaways:

- **Low Swaps:** It performs at most $n-1$ swaps. This is useful for writing to hardware memory where "writes" are "expensive."
- **Simplicity:** Easy to explain and implement in resource-constrained environments.
- **Linearity:** Great for educational purposes to understand $O(n^2)$ behavior.

CODEHIVE ACADEMY